

Test_Lab1

——陈思源（2026 年 10 月 6 日）

P1:

函数要求返回一个最高位为 1 的掩码，只需要把 1 左移 31 位即可，单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f signMask
Score   Rating   Errors   Function
  1      1       0      signMask
Total points: 1/1
```

P2:

函数要求实现两个整数的位异或，而 $x \oplus y = (x | y) \& \sim(x \& y)$ ，由德摩根律知， $x | y = \sim(\sim x \& \sim y)$ ，带入即可实现只用 $\sim$ 与 $\&$ 实现的位异或，有 7 个操作符，单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f bitXor
Score   Rating   Errors   Function
  2      2       0      bitXor
Total points: 2/2
```

P3:

函数要求给定整数为负时返回其相反数，非负时返回 0。利用负数右移 31 位后所有位全为 1 和非负数右移 31 位后全为 0 可实现该函数，单题测试结果如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f negativePart
Score   Rating   Errors   Function
  3      3       0      negativePart
Total points: 3/3
```

P4:

函数要求把目标字节 dst 的数字替换为源字节的数字，现算出得到各自字节的移位量，然后得到源字节和目标字节上的数字，最后进行替换即可，单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f copyByteWithin
Score   Rating   Errors   Function
  4      4       0      copyByteWithin
Total points: 4/4
```

P5:

函数要求右移相应位数，但高位补 0，所以先得到只有前 n 位为 0 的数字，最后与逻辑右移的数字相与即可，单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f logicalShift
Score   Rating   Errors   Function
  4      4       0      logicalShift
Total points: 4/4
```

P6:

函数要求每个字节的高四位与第四位互换，所以先构造掩码 0x0F0F0F0F，然后分别得到每

个字节高 4 位的数字和低 4 位的数字，最后位与即可，单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f swapNibblePairs
Score   Rating  Errors  Function
   4       4       0      swapNibblePairs
Total points: 4/4
```

P7:

函数要求返回标记第二低的零位的位置的掩码，先将第一低的零位变为 1，然后将变化后的数字取反，并与第一次变化后并加 1 的数字进行位与，即可得到对应位标记为 1 的掩码，单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f secondLowestZeroBit
Score   Rating  Errors  Function
   4       4       0      secondLowestZeroBit
Total points: 4/4
```

P8:

函数要求给定数字有奇数个 1 时返回 0，有偶数个 1 时返回 1。由于异或运算满足结合律和交换律，所有位连续异或的结果，就是 1 的个数的奇偶性：如果有奇数个 1，异或和为 1；如果有偶数个 1，异或和为 0。故先用折叠法快速计算所有位的异或和：将高 16 位与低 16 位异或；再将高 8 位与低 8 位异或；依次 4 位、2 位、1 位；最后最低位就是所有位的异或和。然后与 1 位与，最后取反即可，单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f oddParity
Score   Rating  Errors  Function
   5       5       0      oddParity
Total points: 5/5
```

P9:

函数要求把 x 循环右移 n 位。先将 n 对 32 取模，计算实际移位置，然后将循环右移拆成两部分，右移实际移位置（要求高位补 0）后与左移（ $\sim$ 实际移位置 + 1）& 31）取位与即可，单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f rotateRightBits
Score   Rating  Errors  Function
   5       5       0      rotateRightBits
Total points: 5/5
```

P10:

函数要求把非负整数 x 舍入到最近的 2^n 的倍数。如果 x 正好在两个倍数中间，则选择商为偶数的那个倍数。先令 $d = 1 \ll n$ ，即 2^n ； $half = d \gg 1$ ，即 $2^{(n-1)}$ 。然后给 x 加上一个偏置： $bias = half - 1 + (q \& 1)$ ，其中 $q = x \gg n$ 。最后计算 $((x + bias) \gg n) \ll n$ 即可得到结果。单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f roundEvenPow2
Score   Rating  Errors  Function
   5       5       0      roundEvenPow2
Total points: 5/5
```

P11:

函数要求返回 $(x + y) / 2$ 的精确数学中点，且不能溢出。如果中点恰好在两个整数之间（即 x+y 为奇数），则选择靠近第一个参数 x 的那个整数。先用位运算计算 $floor((x+y)/2)$

而不直接相加，避免溢出： $base = (x \& y) + ((x \wedge y) \gg 1)$ ；然后判断 $x+y$ 是否为奇数： $odd = (x \wedge y) \& 1$ ；再判断 $x > y$ 是否成立，结果只取最低位（1 表示真，0 表示假）。利用符号位和同号/异号情况：若 x 和 y 异号， $x > y$ 等价于 x 非负。若同号， $x - y$ 不会溢出，其符号位可判断 $x \geq y$ 。最后，如果 $x+y$ 是奇数且 $x > y$ ，则结果需要在 $base$ 上加 1。单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f midpointTowardFirst
Score   Rating  Errors  Function
   5       5      0    midpointTowardFirst
Total points: 5/5
```

P12:

函数要求若 x 在区间 a, b 里则返回 1，反之则返回 0。

关键：无溢出判断 $x \geq y$ 。

先用 $diff = x + \sim y + 1$ 表示 $x - y$ ，再用掩码修正符号位： $E = diff \wedge ((x \wedge y) \& (diff \wedge x))$ ； E 的符号位为 0 表示 $x \geq y$ ，为 1 表示 $x < y$ 。然后用 $\sim(E \gg 31)$ 得到全 1（真）或全 0（假）。计算四个比较：

$x_ge_a: x \geq a$

$x_ge_b: x \geq b$

$a_ge_x: a \geq x$ ，即 $x \leq a$

$b_ge_x: b \geq x$ ，即 $x \leq b$

组合条件： $(x_ge_a \& b_ge_x) \mid (x_ge_b \& a_ge_x)$

最后 $\& 1$ 归一化为 0 或 1。单题测试结果如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f isBetweenEitherOrder
Score   Rating  Errors  Function
   7       7      0    isBetweenEitherOrder
Total points: 7/7
```

P13:

函数要求计算 $x*5$ ，如果溢出则饱和到 INT_MAX 或 INT_MIN 。

先用移位和加法计算 $x*5 = (x \ll 2) + x$ 。

分两步检查溢出：

先检查 $x \ll 2$ 是否溢出：如果 $(x \ll 2) \gg 2 \neq x$ ，则左移溢出。

再检查加法 $x4 + x$ 是否溢出：如果 $x4$ 和 x 同号，但结果 $result$ 与它们异号，则加法溢出。

如果任意一步溢出，根据 x 的符号返回 INT_MAX 或 INT_MIN 。

最后用掩码选择饱和值或计算结果。单题测试结果如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f mul5Sat
Score   Rating  Errors  Function
   7       7      0    mul5Sat
Total points: 7/7
```

P14:

函数要求判断 $x + y + z$ 的精确数学和是否超出 int 范围，返回 1、-1 或 0。

先提取 x, y, z 的符号位（算术右移 31 位）。然后算 $x + y$ ，得到低 32 位 $s1$ 和进位

c_1 (进位用位运算公式计算, 并 $\& 1$ 归一化为 0 或 1), 高 32 位 $h_1 = h_x + h_y + c_1$ 。
再算 $s_1 + z$, 得到低 32 位 s_2 和进位 c_2 , 高 32 位 $h_2 = h_1 + h_z + c_2$ 。
判断溢出: 如果 h_2 等于 s_2 的符号扩展 (即 $h_2 == (s_2 \gg 31)$), 说明结果在 `int` 范围内, 返回 0; 否则溢出。
返回溢出方向: 溢出时, 若 h_2 的符号位为 0 (正溢出), 返回 1; 为 1 (负溢出), 返回 -1。单题测试结果如下:

```
drink_water@localhost:~/Test_Lab1$ ./btest -f classifyAdd3
Score   Rating  Errors  Function
   7       7      0    classifyAdd3
Total points: 7/7
```

P15:

函数要求计算浮点数 $f * 1.5$, 输入输出都是 `unsigned int` 表示的单精度浮点数值模式。需要处理规格化数、非规格化数、零、无穷大和 NaN, 并使用 `round-to-nearest-even` 舍入。大致思路:

1. 分解浮点数的符号、阶码、尾数。
2. 处理特殊值 (NaN、无穷、零)。
3. 将浮点数转换为有效数字和实际指数。
4. 乘以 3, 再除以 2 (通过调整指数)。
5. 重新规格化, 判断结果是非规格化、规格化还是溢出。
6. 使用 `round-to-nearest-even` 舍入规则。
7. 组装并返回结果位模式。

单题测试如下:

```
drink_water@localhost:~/Test_Lab1$ ./btest -f floatScaleThreeHalves
Score   Rating  Errors  Function
   7       7      0    floatScaleThreeHalves
Total points: 7/7
```

P16:

函数要求把单精度浮点数舍入到最近的整数, 中点舍入到偶数, 并返回整数的单精度浮点表示。大致思路如下:

1. 拆分输入

从 `uf` 中取出符号位 s 、阶码 e 、尾数 m 。

2. 处理特殊值

$e == 0xFF$: NaN 或无穷大, 直接返回 `uf`。

$e == 0 \ \&\& \ m == 0$: ± 0 , 直接返回 `uf`。

3. 根据阶码判断数值范围

$e == 0$: 非规格化数, 绝对值小于 0.5, 舍入到 0, 返回符号 s 。

$e < 126$: 绝对值小于 0.5, 舍入到 0, 返回 s 。

$e == 126$: 值在 $[0.5, 1)$ 之间。

$m == 0$ 表示正好 0.5, 向偶数舍入到 0, 返回 s 。

$m != 0$ 表示大于 0.5, 舍入到 1, 返回 1.0 的浮点表示。

$e \geq 150$: 数值已经大于等于 2^{23} , 没有小数部分, 直接返回 `uf`。

$127 \leq e < 150$: 有小数部分, 需要做 `round-to-nearest-even` 舍入。

4. 舍入有小数的情况

计算需要右移的位数: $shift = 150 - e$ 。

构造 24 位有效数字: $\text{sig} = (1 \ll 23) \mid m$ 。

整数部分: $\text{int_part} = \text{sig} \gg \text{shift}$ 。

小数部分: $\text{frac} = \text{sig} \& ((1 \ll \text{shift}) - 1)$ 。

中点: $\text{half} = 1 \ll (\text{shift} - 1)$ 。

若 $\text{frac} > \text{half}$, 或 $\text{frac} == \text{half}$ 且 int_part 为奇数, 则 $\text{int_part}++$, 实现向偶数舍入。

5. 重新编码为单精度浮点数

若 $\text{int_part} == 0$, 返回符号 s (表示 ± 0)。

找 int_part 的最高位 p , 新阶码 $e_{\text{new}} = p + 127$ 。

根据 int_part 构造新的尾数 m_{new} 。

返回 $s \mid (e_{\text{new}} \ll 23) \mid m_{\text{new}}$ 。

单题测试如下:

```
drink_water@localhost:~/Test_Lab1$ ./btest -f floatRoundEven
Score   Rating  Errors  Function
   10     10      0    floatRoundEven
Total points: 10/10
```

P17:

函数要求把 32 位有符号整数 x 转换为单精度浮点数的位模式, 返回 unsigned int。

大致思路如下:

1. 处理 0: 直接返回 0 (即 +0.0)。

2. 处理符号: 若 $x < 0$, 符号位 $s = 0x80000000$, 并取绝对值。注意 INT_MIN 的绝对值是 2^{31} , 用无符号取反加一可正确处理。

3. 找最高位 p : 确定 u 中最高位 1 的位置, p 从 31 递减。

4. 计算阶码: 规格化后实际指数为 p , 阶码 $e = p + 127$ 。

5. 构造尾数:

若 $p \leq 23$, 尾数直接左移 $(23 - p)$ 位。

若 $p > 23$, 右移 $\text{shift} = p - 23$ 位, 并做 round-to-nearest-even 舍入:

取整数部分 $\text{sig} = u \gg \text{shift}$, 丢弃位 $\text{dropped} = u \& ((1 \ll \text{shift}) - 1)$, 中点 $\text{half} = 1 \ll (\text{shift} - 1)$ 。

若 $\text{dropped} > \text{half}$, 或 $\text{dropped} == \text{half}$ 且 sig 为奇数, 则 $\text{sig}++$ 。

若 sig 进位到 $1 \ll 24$, 则 $\text{sig} \gg= 1$, $e++$ 。

6. 组装结果: $s \mid (e \ll 23) \mid (\text{sig} \& 0x7FFFFF)$ 。

单题测试结果如下:

```
drink_water@localhost:~/Test_Lab1$ ./btest -f float_i2f
Score   Rating  Errors  Function
   10     10      0    float_i2f
Total points: 10/10
```

P18:

函数要求统计 x 的二进制表示中 1 的个数。

思路: 采用分治法 (并行加法), 逐步将相邻位的 1 的个数合并。

每 2 位一组, 计算每组中 1 的个数, 结果存回这 2 位。

每 4 位一组, 合并相邻 2 位的计数。

每 8 位、16 位、32 位依次合并。

最终得到 32 位中所有 1 的总数。

单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f bitCount
Score  Rating  Errors  Function
   10    10     0     bitCount
Total points: 10/10
```

P19:

函数要求反转 32 位整数的所有位。使用分治交换法，依次交换相邻的 1 位、2 位、4 位、8 位和 16 位。由于整数常量限制在 0x00 - 0xFF，需要用移位和或运算构造所需的 32 位掩码。单题测试如下：

```
drink_water@localhost:~/Test_Lab1$ ./btest -f bitReverse
Score  Rating  Errors  Function
   10    10     0     bitReverse
Total points: 10/10
```

在终端中执行 `./check_ops.py bits.c` 后的截图：

```
drink_water@localhost:~/Test_Lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 6/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 10/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 40/48 operators
mul5Sat: 25/30 operators
classifyAdd3: 32/52 operators
floatScaleThreeHalves: 58/60 operators
floatRoundEven: 40/65 operators
float_i2f: 35/40 operators
bitCount: 24/40 operators
bitReverse: 33/34 operators
All 19 functions passed operator checks.
```

在终端中执行 `./btest` 后的截图：

```
drink_water@localhost:~/Test_Lab1$ ./btest
```

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse

Total points: 110/110